

ProbOS — Month 2 Plan

Nisong Monyimba

July 2, 2026

Theme

Sequential inference plus bringing the C++ kernel into the Python package. Month 1 built a forward-simulation kernel: draw priors, propagate, summarise, explain. Month 2's central gap is **inference** — given observed data, update beliefs about latent state and parameters online. That capability is the particle filter, and it is the spine around which every other Month 2 workstream is organised.

Week 5 — Particle Filter Core

Goal: a working sequential Monte Carlo filter, validated against a known analytical case before touching `BatteryModel2Cell`.

- `python/src/particle_filter.py`
 - `ParticleFilter` class: init from prior, `predict()` step (reuses `Model.forward_batch()`), `update()` step (likelihood weighting), `resample()` step (systematic resampling)
 - Effective sample size (ESS) diagnostic, resample-when-ESS-low trigger
- Validate against a linear-Gaussian model with a known Kalman filter solution first, then apply to `BatteryModel2Cell` with synthetic noisy temperature observations.
- Study guide: Chopin & Papaspiliopoulos (2020) Ch 8–10 on resampling schemes; Naesseth et al. (2019) for the unifying SMC algorithm template.

Exit test: filter posterior mean tracks true latent state within theoretical bounds on the linear-Gaussian case; ESS never collapses without triggering resampling.

Week 6 — pybind11: C++ Kernel Enters the Python Package

Goal: stop maintaining two parallel implementations. Bind `BatteryCell` and `MonteCarloEngineOMP` into Python via `pybind11`.

- `cpp/bindings/` — `pybind11` module definition
- `setup.py` / `CMakeLists.txt` updates for building the extension
- Cross-validation test: C++ and Python engines must agree on percentiles for the same seed within floating-point tolerance

Exit test: `import probos.cpp` works from a plain `pytest` run in CI.

Week 7 — FastAPI Service Layer

Goal: expose the kernel over HTTP.

- POST `/simulate` — run `MonteCarloEngine`, return percentiles and convergence summary
- POST `/sensitivity` — run `SobolSensitivity`, return S1/ST indices
- POST `/filter` — run `ParticleFilter` against posted observation data
- Pydantic schemas mirroring the `Model/Distribution` constructor signatures

Exit test: a curl POST to `/simulate` returns valid JSON matching a direct Python call to the same `MonteCarloEngine`.

Week 8 — Cross-Discipline Batch 2 + Month 2 Retrospective

Goal: show the particle filter also generalises across domains.

- Port the ED queue model to a filtering setting (infer true arrival rate from observed queue-length time series)
- Port the clinical trial model to sequential monitoring, using the particle filter instead of batch nested-MC
- `docs/retrospectives/month2_retrospective.md` plus `month3_plan.md`

Exit test: both filtering examples pass validation against ground truth, with test coverage matching Month 1's standard.

Standing Month 2 commitments (every week)

- Run `check_ci.sh` after every push, no exceptions
- Update `docs/study/study_guide.md` same-session as any new file needing external reading
- mypy strict and ruff clean before any commit
- No claimed speedup or result without a genuine benchmark backing it